

이 코드, 어디서 왔니?

AI 생성 코드의 계보 기반 신뢰 추적 시스템

목차

□ Chapter 1 문제 정의 - 바이트 코딩 시대의 도래 - 기술적 실독증의 양산 - 소프트웨어 생애 주기 전반의 위험	□ Chapter 5 실제 적용 방식 - 개발 단계: IDE 플러그인 - 통합 단계: Git PR 자동 검증 - 운영 단계: 취약점 전파 경로 분석
□ Chapter 2 원인 분석 - AI 역량 불균형 - 사고 과정 소실 - 기존 도구의 구조적 제약 - 검증 체계 미비	□ Chapter 6 기대 효과 - 장애 원인 파악 시간 단축 - 미검토 AI 코드 가시화 - 보안 취약점 사전 차단 - 유지보수 효율 향상
□ Chapter 3 핵심 아이디어 - 생성 맥락 서명(GCA) - 세션 단위 계보 추적 - 위험 전파 경로 분석 - 설명 가능한 신뢰점수	□ Chapter 7 한계점 - 초기 데이터 수집 문제 - AST 추적 기술적 한계 - GCA 생태계 협력 의존 - 신뢰 점수 오남용 가능성
□ Chapter 4 기술 아키텍처 - 코드 블록별 출처 추적 - Git 커밋·AST 결합 분석 - 위험 전파 그래프 모델 - 신뢰 점수 산출법	□ Chapter 8 확장 가능성 - 멀티 에이전트 환경 확장 - AST 추적 기술적 한계 - EU AI Act 컴플라이언스 - AI 산출물 전반으로 확장

문제 정의

바이브 코딩 시대의 도래

- 생성형 AI의 급속한 발전으로 소프트웨어 개발 진입 장벽 근본적 하락
- ChatGPT, Claude, GitHub Copilot 등 바이브 코딩 도구의 확산
- 전문 지식 없이도 즉시 코드 생성 가능한 환경 조성
- 코딩은 쉬워지고 속도는 빨라진 시대

근본적 질문의 등장

- '이 코드는 어디서 왔는가?'
- '왜 이렇게 작성되었는가?'
- '이 코드를 신뢰할 수 있는가?'
- 출처와 신뢰성에 대한 근본적 의문 발생

편리함의 이면

- 속도와 생산성이라는 단기적 이득
- 점점 잊혀가는 중요한 질문들
- 코드의 출처와 작성 이유에 대한 무관심
- 신뢰 기반의 약화

시스템 기반의 위험

- 운영 시스템의 기반이 서서히 흔들림
- 검증되지 않은 코드의 누적
- 장기적 안정성과 유지보수성 저하
- 소프트웨어 품질 전반의 위협

핵심 문제 (1): 기술적 실독증의 양산

코드 이해의 단절

- AI 생성 코드의 실행 여부만 확인하는 개발자
- 내부 로직 흐름과 설계 의도 파악 불가
- '기술적 실독증(Technical Illiteracy)' 상태 유발
- 자신이 운영하는 시스템의 작동 원리 미이해

검증 없는 수용

- AI는 프롬프트라는 좁은 맥락 안에서만 사고
- 할루시네이션으로 인한 결함 코드 생성 가능
- 겉보기 완벽하지만 내부적 심각한 결함 포함
- 검증 능력 결여 상태에서 서비스 반영

이력과 책임의 상실

- 코드 변경 사항은 기록되지만 맥락은 사라짐
- 어떤 의도(Prompt)로 생성되었는지 추적 불가
- AI와 인간 중 누구의 판단인지 불명확
- 코드는 남지만 맥락은 상실

기존 도구의 한계

- Git은 변경 사항만 기록, 계보는 추적 못함
- 코드 생성 의도와 AI 모델 정보 부재
- 정적 분석 도구의 현재 상태 중심 분석
- 생애 주기 추적 능력 결여

핵심 문제 (2): 소프트웨어 생애 주기 전반의 위험

품질 저하

- AI는 로컬 로직 최적화에만 집중
- 프로젝트 전체 아키텍처 무시
- 기존 코드 컨벤션과 충돌 발생
- 중복 로직 반복 생성으로 구조적 품질 하락

안정성 문제

- 명시하지 않은 예외 상황 처리 생략
- 네트워크 지연, 비정상 입력 등 미고려
- 개발자가 인지하지 못한 채 배포
- 특정 조건에서 시스템 예기치 않게 중단

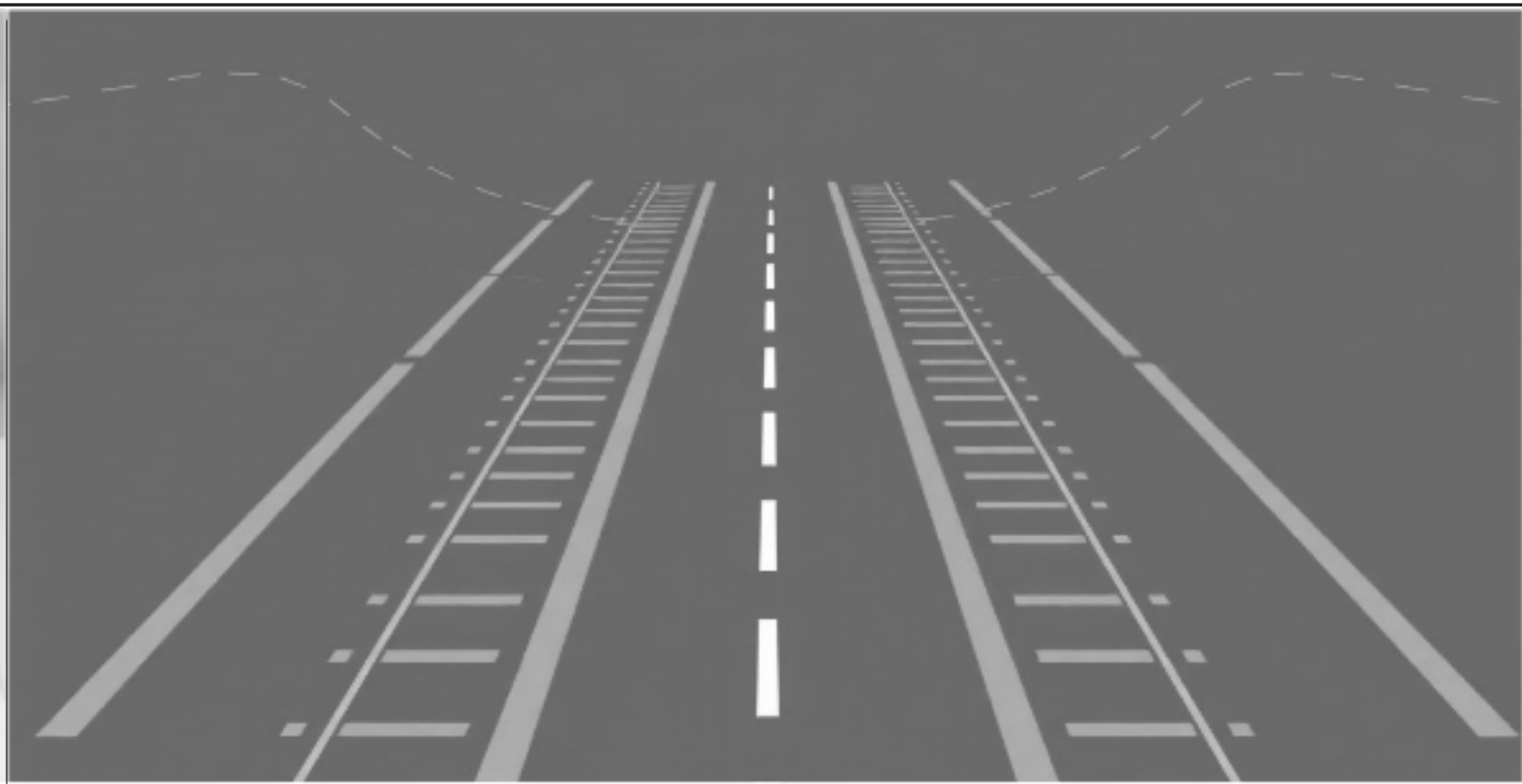
보안 취약점

- 학습 데이터의 구시대적 보안 패턴 포함
- 취약한 라이브러리 로직 필터링 없이 삽입
- 코드 해석 불가능한 개발자
- 보안 사고 발생 전까지 위험 인지 불가

유지보수 불가능

- 장애 발생 시 작성 주체와 맥락 불명
- 원인 파악에 과도한 시간 소요
- '무서워서 건드리지 못하는 코드' 누적
- 시스템 전체가 거대한 레거시 덩어리로 굳어감

문제의 본질: 신뢰 패러다임의 전환 필요성



기존 신뢰 근거

- '이해'에 기반한 개발 문화
- 개발자가 직접 코드 작성
 - 동료의 코드 리뷰
- 주석과 커밋 메시지로 의도 기록
- 일련의 과정이 신뢰의 근거

AI 시대의 간극

- AI가 코드 생산하는 속도
- 인간이 검증하는 속도
- 두 속도 사이의 간극 확대
- 코드 이해 없이도 개발 가능
- 출처와 맥락 추적 불가 시 신뢰 불가능

새로운 신뢰 근거

- '검증과 추적'으로의 전환
- 코드 출처와 맥락 추적 필수
- 이해에서 검증으로 신뢰 근거 이동
- 추적 가능한 근거 기반 신뢰 체계
- 계보 기반 분석의 필요성

원인 분석 (1): AI 역량의 비대칭성

생산 속도의 혁신적 향상

- AI는 수조 개의 데이터를 학습해 언어와 프레임워크를 가리지 않고 즉시 코드 생성
- 인간이 소요하던 시간을 초 단위로 단축
- 코드 생산 능력의 비약적 발전

프롬프트 기반의 제한된 맥락 이해

- AI는 사용자가 제공한 프롬프트라는 좁은 틀 안에서만 사고
- 실제 서비스의 복잡한 비즈니스 로직 파악 불가
- 숨겨진 예외 상황과 시스템 전체 맥락 이해 한계

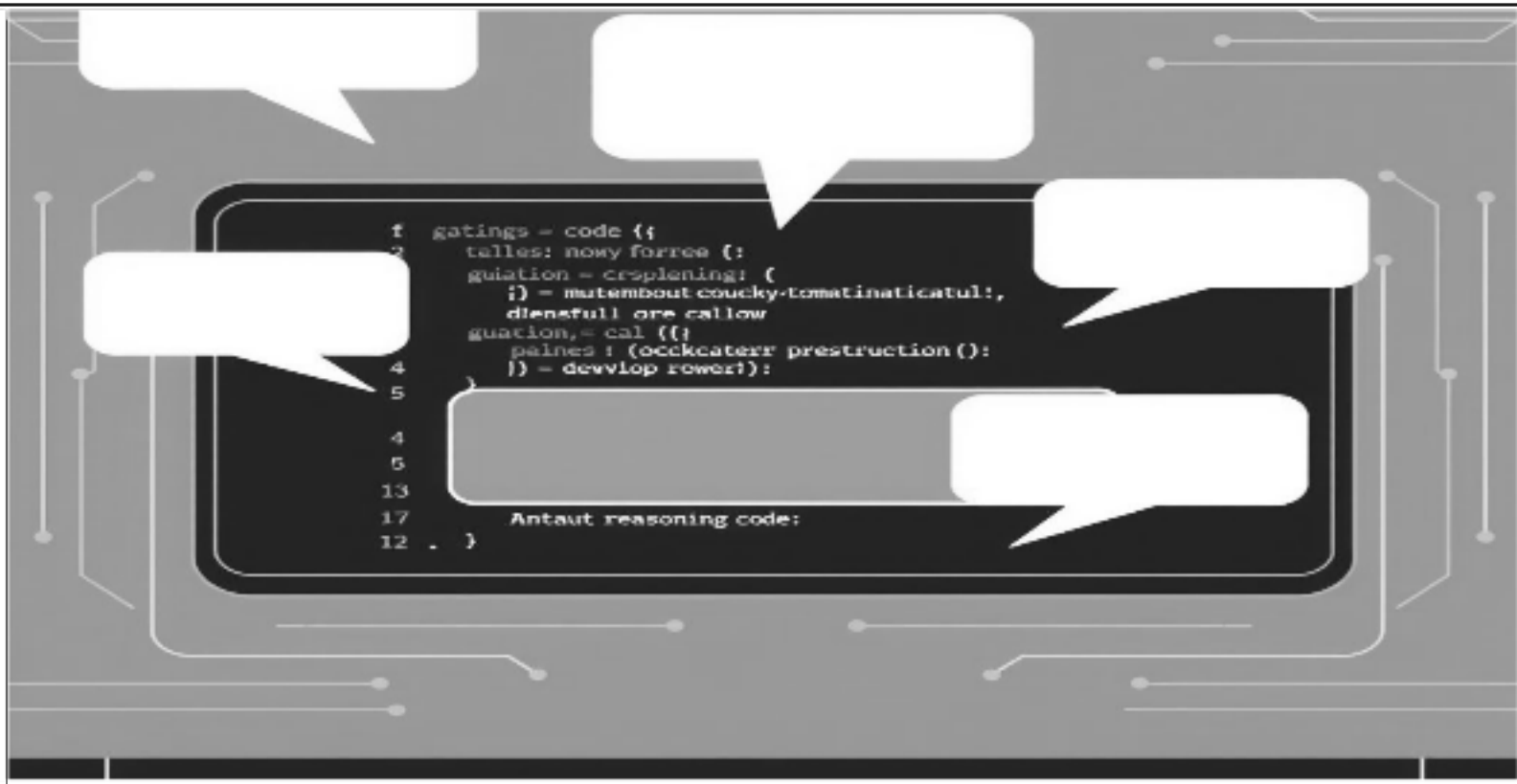
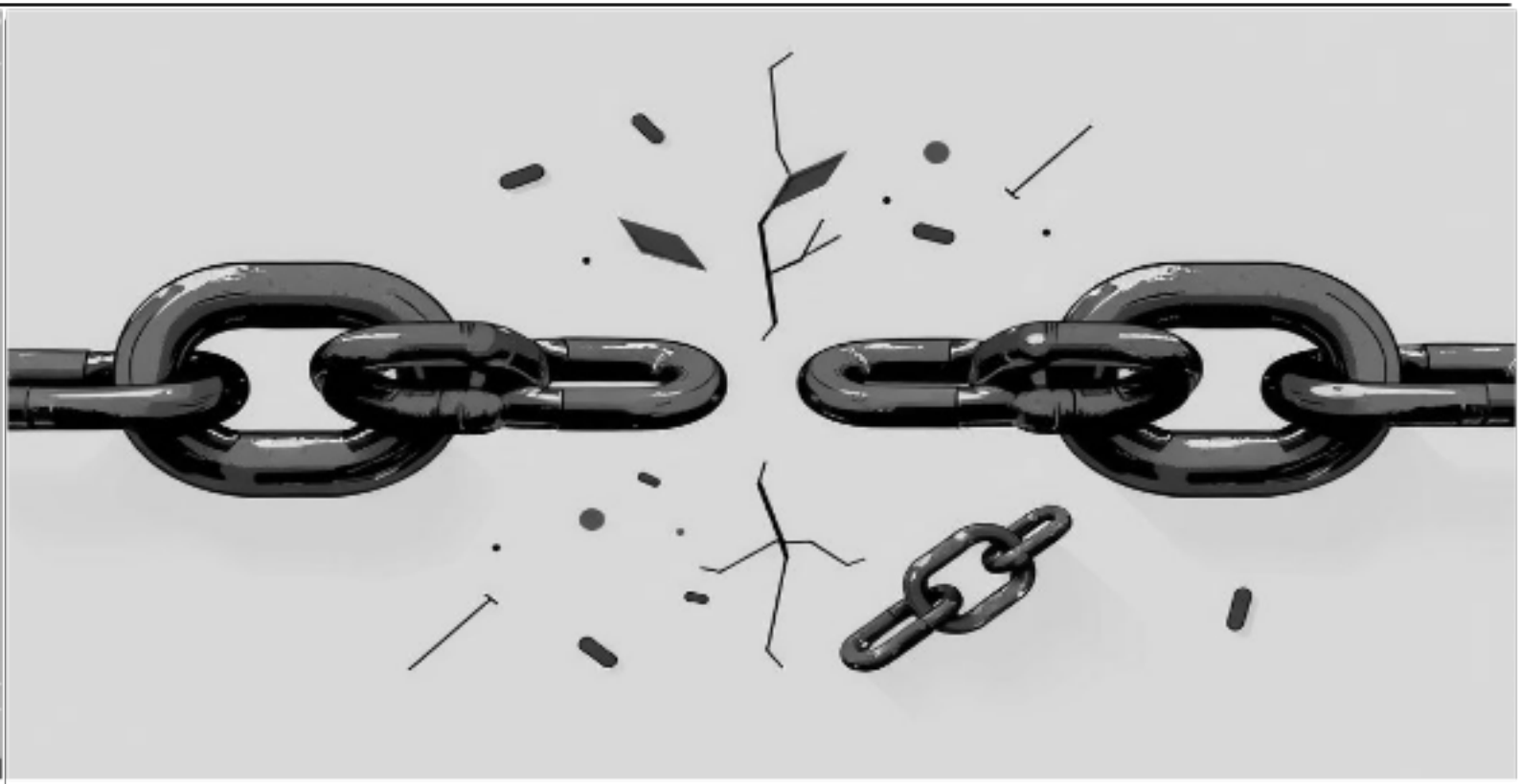
비즈니스 로직과 예외 상황 파악 한계

- 논리적 완결성 판단은 여전히 인간의 영역
- 실제 서비스 환경의 복잡성 고려 부족
- 예외 처리와 엣지 케이스 대응 미흡

속도가 검증을 앞지르는 구조적 문제

- 개발자는 AI의 빠른 속도에 이끌려 논리적 결함 검토 전에 코드 통합
- 속도가 검증을 앞질러 버리는 구조
- 생산과 검증 사이의 간극 확대

원인 분석 (2): 사고 과정의 소멸

		
기존 개발 방식	AI 생성 코드	유지보수 단서 상실
- 개발자가 고민한 흔적이 주석으로 기록 - 시행착오가 커밋 메시지로 남음 - 판단의 근거가 코드 리뷰로 공유 - 코드는 단순한 결과물이 아닌 사고의 기록	- AI는 최종 코드 블록만 제공 - 왜 이 방식을 선택했는지 알 수 없음 - 보안 요소 고려 사항이 블랙박스 안에 사라짐 - 추론 과정이 완전히 생략됨	- 코드 수정 시 누가 왜 바꿨는지 인과관계 단절 - 유지보수 시 코드 해석 단서 부재 - 코드는 남지만 맥락은 사라지는 구조 - 장애 발생 시 원인 파악 불가능

원인 분석 (3)

- ✓ 정적 분석 도구와 Git의 Snapshot 중심 분석



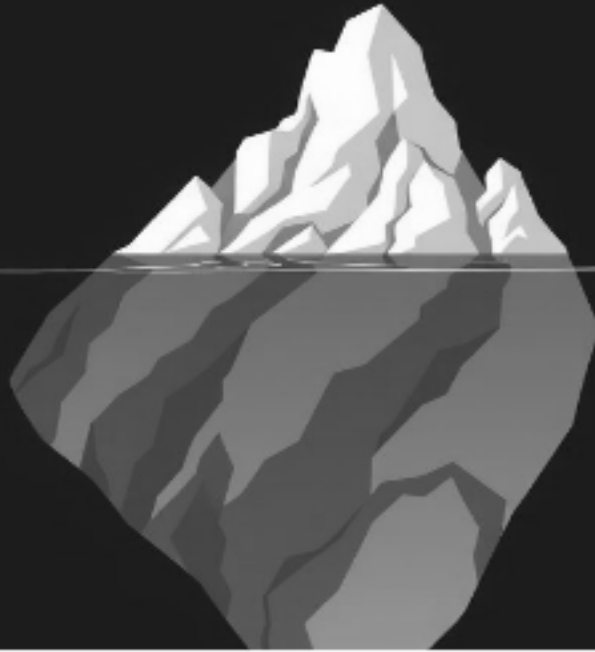
- 현재 상태(Snapshot)를 진단하는 데 최적화
- 구문 오류, 스타일 체크, 변경 이력만 기록
- 코드가 어떻게 생성되었는지 알 수 없음
- 코드의 현재 모습만 분석 가능
- 생성 과정과 의도는 추적 불가

- ✓ 바이브 코딩 환경의 Lineage 추적 요구사항

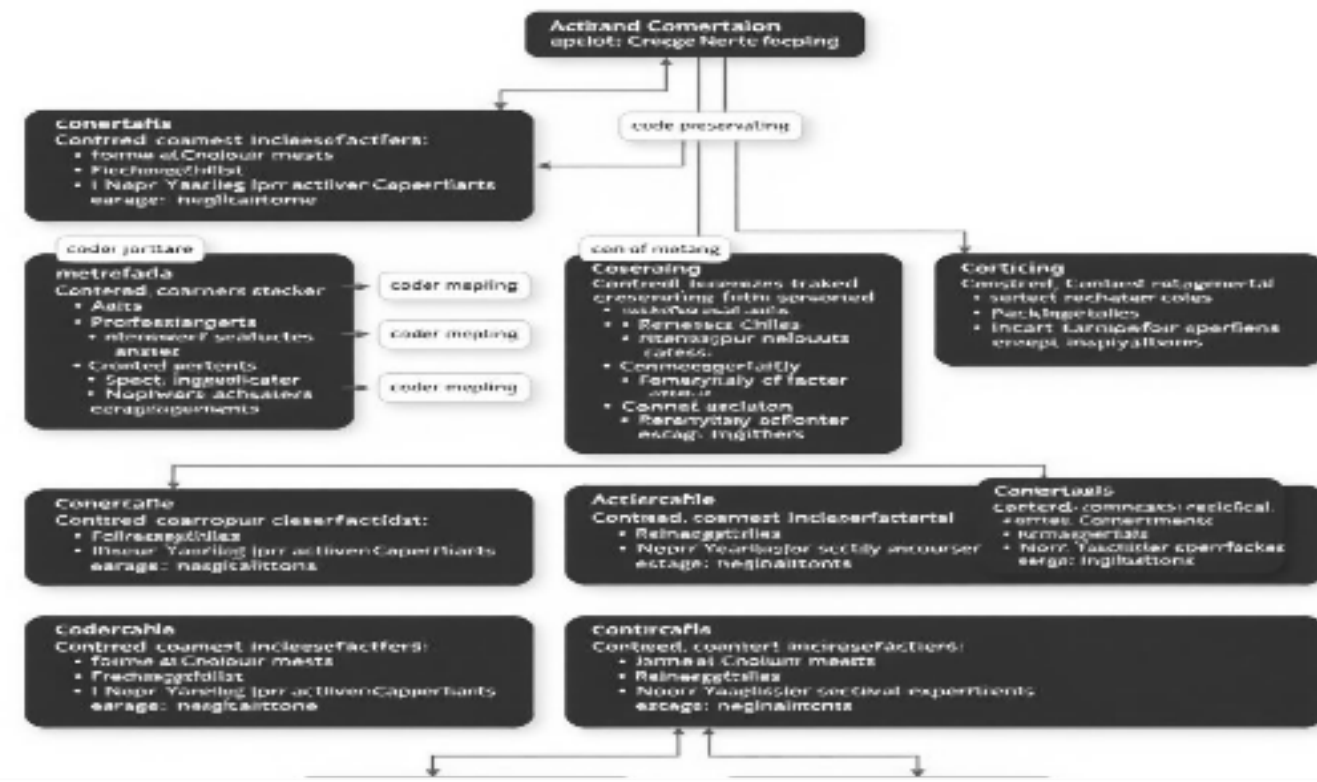


- 코드의 생애 주기(Lineage)를 거슬러 올라가는 추적 능력 필요
- 코드 출처, 생성 의도(Prompt), 모델 신뢰도 정보 요구
- 코드의 뿌리부터 신뢰 문제 해결 가능한 체계
- 생성 맥락과 전파 경로 추적 필수

원인 분석 (4): 검증 체계의 부재

		
실행 성공의 착각	보안과 품질 미보장	필터링 없는 수용
- 에러 없이 실행되는 코드를 좋은 코드로 간주하는 위험 - 동작하는 코드 생성에만 최적화된 AI - 실행 성공이 품질 보증은 아님 - 표면적 정상 작동의 함정	- 보안 안전성 보장 없음 - 예외 강건성 검증 부재 - 자원 효율성 고려 미흡 - 독립적 검증 자동화 레이어 부재 - 비기능적 요구사항 충족 불확실	- 개발자는 AI 결과물을 필터링 없이 수용 - 자동화된 검증 체계 부재 - 코드 품질 검증 단계 생략 - 시스템 전체 신뢰도 저하

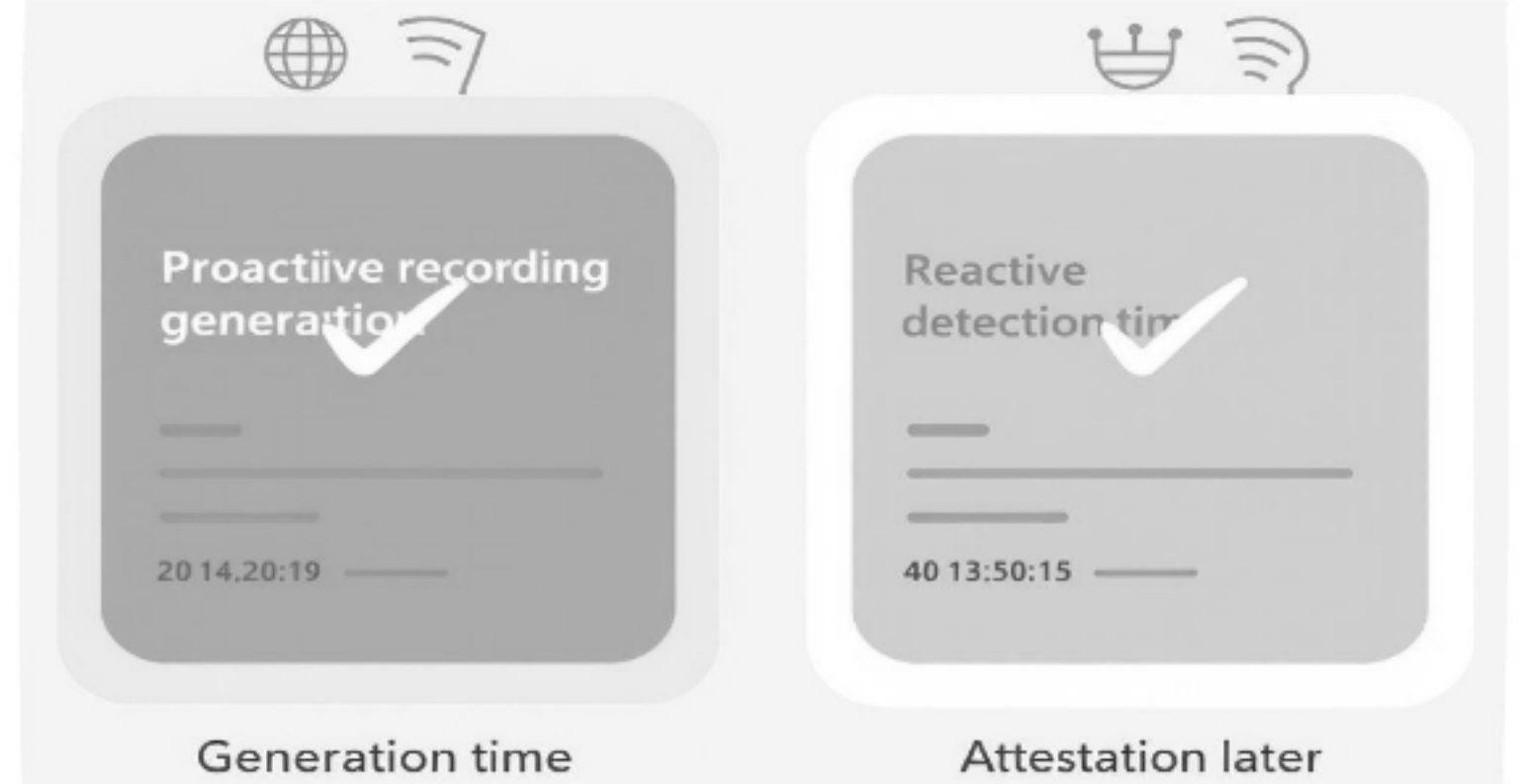
핵심 아이디어: 계보 기반 신뢰 추적



전 과정 추적



맥락 기록



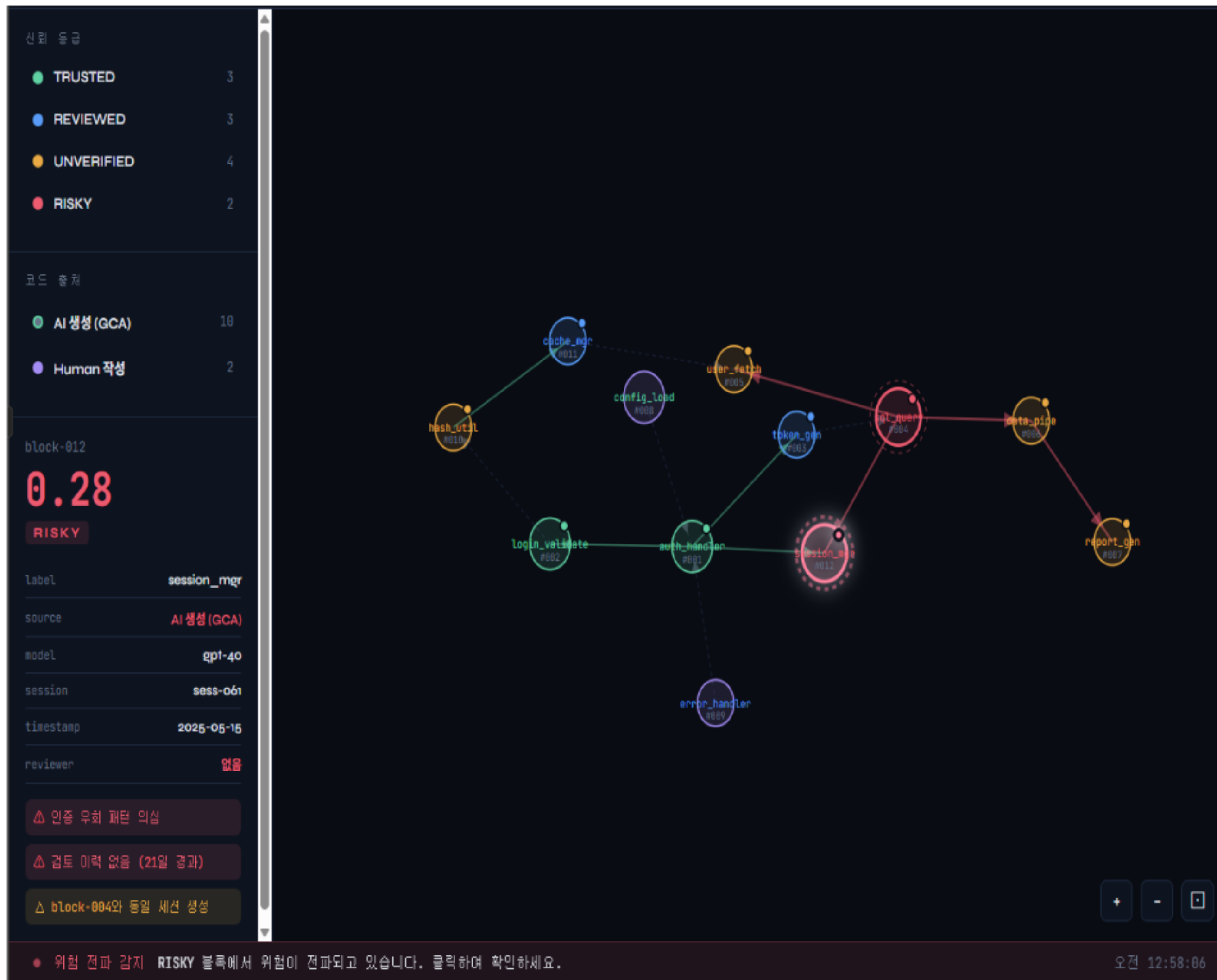
기록 중심 접근

- 코드가 태어나는 순간부터 수정되고 전파되는 전 과정 추적
 - 생성, 수정, 전파의 완전한 이력 관리
 - 시간의 흐름에 따른 코드 변화 기록
 - 생애 주기 전체의 투명성 확보

- 맥락을 추적 가능한 형태로 기록
 - 코드의 출처와 생성 의도 보존
- 신뢰의 근거로 활용하는 체계 구축
- 검증 가능한 증거 기반 신뢰 체계

- 감지(Detection)가 아닌 기록(Attestation) 방식
 - 생성 시점부터 메타데이터 암호학적 서명
 - 사후 감지의 불확실성 제거
 - 능동적 추적 체계 구축

계보 기반 그래프: 신뢰 추적 시각화



계보 그래프 시각화

- 코드 블록의 생성부터 전파까지 전 과정을 노드와 엣지로 표현한 그래프이다.
- 각 노드는 신뢰 등급에 따라 색상으로 구분된다.
- 엣지는 색상에 따라 파생, 의존, 전파 등 관계를 표현한다.
- 그래프는 위에서 아래 방향으로 흐른다. 상단에 위치한 노드일수록 계보의 출발점에 가깝다.
- 노드에는 해당 블록 GCA 서명 정보, 생성 세션, 검토 이력 등 확인 가능하다.

핵심 아이디어 (1): 생성 맥락 서명(GCA)

암호학적 서명 부여

- AI가 코드를 생성하는 순간 생성 맥락 서명(GCA) 부여
- 암호학적으로 코드에 메타데이터 결합
- 위조 불가능한 서명 체계
- 생성 시점의 완전한 기록

생성 정보 기록

- 어떤 모델이 생성했는지 기록 (model_id)
- 어떤 프롬프트 의도로 만들었는지 기록 (prompt_hash)
- 언제 생성되었는지 기록 (generation_timestamp)
- 세션 정보와 원본 블록 연결

코드 블록 식별

- AST(Abstract Syntax Tree) 기반 코드 블록 식별
- 구조적 동일성 기준으로 블록 구분
- 변수명, 공백 등 의미 없는 요소 제거 후 해시 생성
- 고유한 block_id 부여

출생 증명서 발급

- 코드의 출생 증명서 역할
- 감지가 아닌 기록 방식의 핵심
- 생성 맥락의 영구적 보존
- 추적 가능한 신뢰의 시작점

핵심 아이디어 (2): 세션 단위 계보 추적



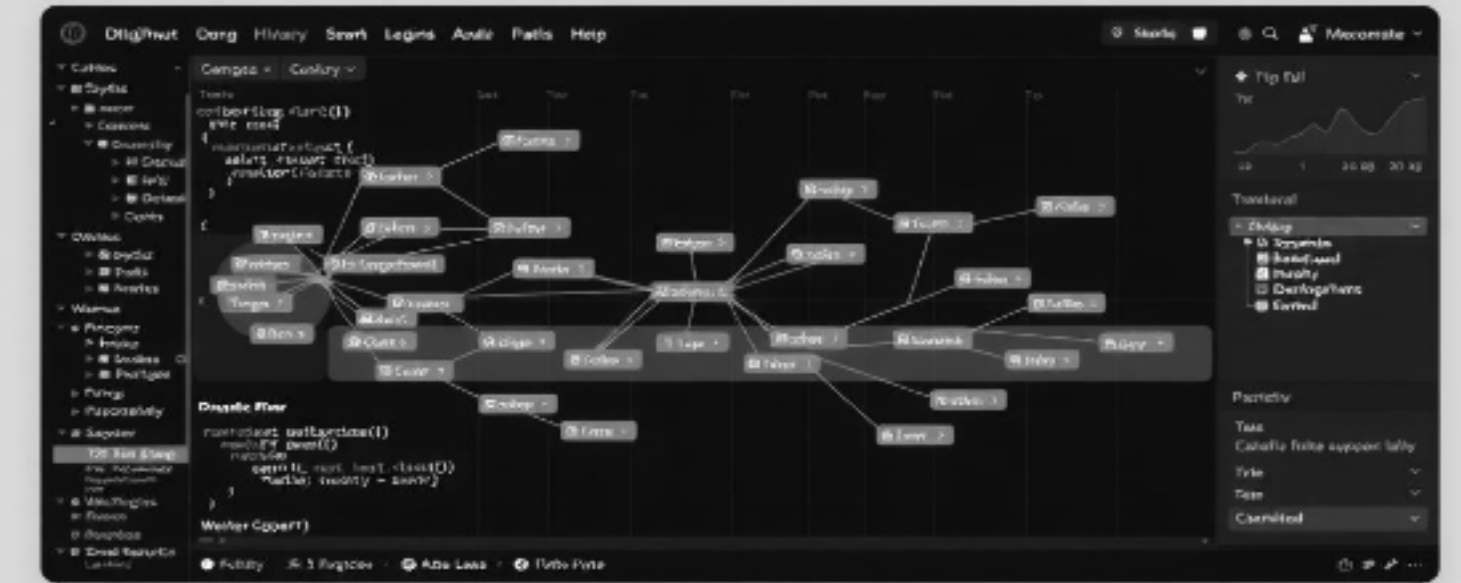
바이브 코딩 세션 추적 단위화

- AI와의 한 세션을 추적 단위로 정의
- 세션 ID로 코드 생성 맥락 그룹화
- 동일 세션의 모든 코드 블록 연결 추적
- 세션 단위로 개발 의도와 목적 기록



프롬프트-코드 인과관계 연결

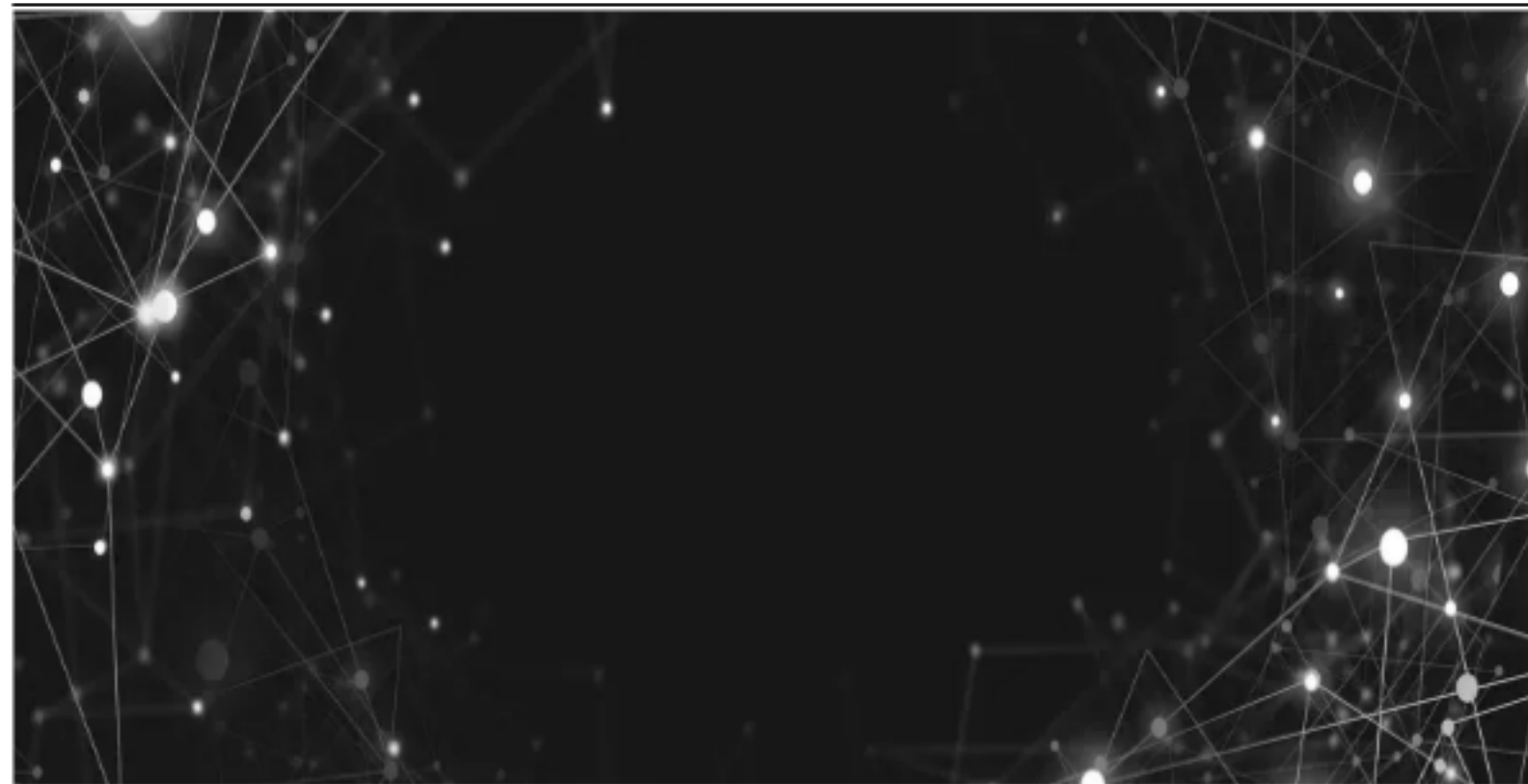
- 프롬프트 해시와 코드 블록 직접 연결
- AI 응답과 적용 코드의 인과 명확화
- 수정 시에도 원본 프롬프트 추적 가능
- 코드 변경 이유를 프롬프트 이력으로 역추적



세션 계보 그래프 시각화

- 버그 발생 시 원인 세션 즉시 파악 가능
- '5월 20일 인증 기능'처럼 구체적 추적
- 세션 그래프로 코드 전파 경로 시각화
- 유지보수 시 생성 맥락 빠르게 이해

핵심 아이디어 (3): 위험 전파 경로 분석

		
취약점 의존 블록 전파 추적	계보 그래프 기반 위험 점수	모델 수준 위험 프로파일
- 특정 코드 블록의 취약점이 의존 블록으로 자동 전파 - 방향성 비순환 그래프(DAG)를 통한 영향 범위 추적 - 직접 의존과 간접 의존 관계를 모두 분석 - 취약점 발견 시 영향받는 모든 블록 즉시 식별	- 전파 단계별로 위험 점수 자동 계산 (decay=0.7) - 0단계: 1.0, 1단계: 0.70, 2단계: 0.49로 감소 - 의존 깊이에 따른 위험도 정량화 - 전파 경로를 시각화하여 영향 범위 명확화	- 동일 AI 모델이 반복 생성하는 취약 패턴 클러스터링 - AST 패턴 분석으로 유사 코드 블록 그룹화 - 모델 버전별 취약점 발생 이력 데이터베이스 구축 - 특정 모델의 구조적 문제를 사전에 경고

핵심 아이디어 (4): 설명 가능한 신뢰 점수

GCA 서명 유효성 + 모델 신뢰 등급

- 암호학적 서명 검증을 통한 코드 출처 확인
- AI 모델별 과거 취약점 발생 이력 기반 신뢰도 평가
- 두 요소를 결합하여 기본 신뢰 수준 산출
- 서명 위조나 미검증 모델 사용 시 즉시 감지

사람 검토 이력 + 전파 위험도

- 코드 리뷰 완료 여부 및 검토자 수 반영
- 검토 시점과 최신성을 고려한 가중치 적용
- 의존 블록에서 전파된 위험 점수 통합
- 네 가지 요소의 가중 합산으로 최종 점수 계산

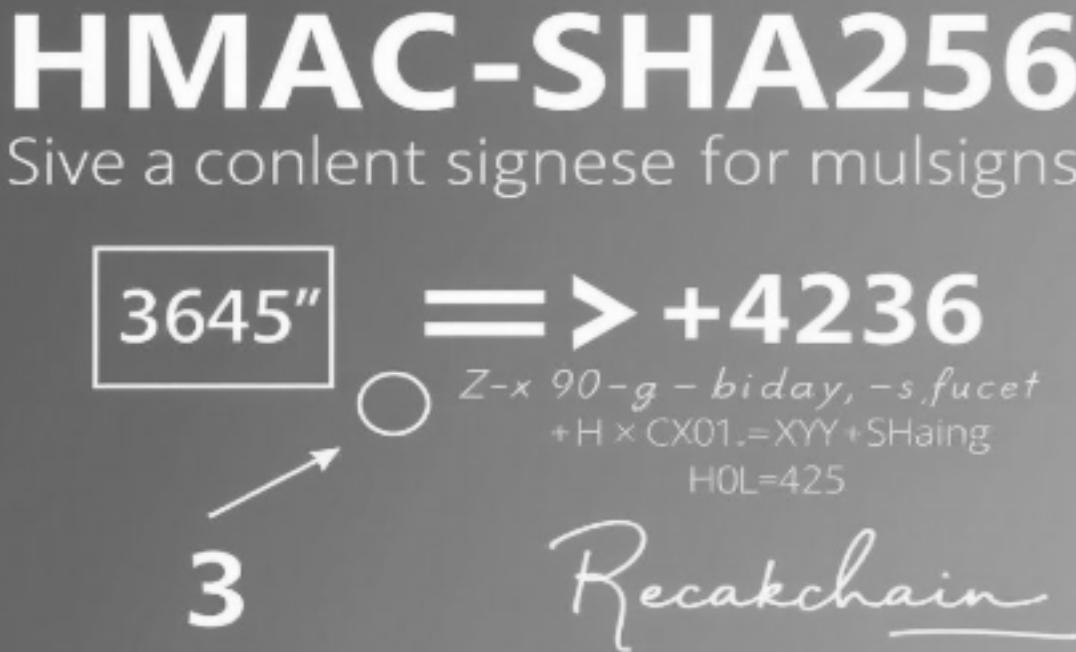
숫자가 아닌 맥락 제공

- 단순 숫자 점수가 아닌 상세한 설명 컨텍스트 제공
- 서명 상태, 모델 이력, 검토 여부, 위험 전파 경로 명시
- 각 요소별 구체적인 근거와 권장 조치 포함
- 개발자가 신뢰 판단의 이유를 명확히 이해 가능

추적 가능한 근거 기반 체계

- 실행 성공 여부가 아닌 추적 가능한 근거 중심
- 코드 생성부터 검토까지 전 과정 투명하게 기록
- 책임 소재를 명확히 하는 구조적 신뢰 체계
- 장기적 유지보수와 규제 대응을 위한 증거 확보

기술 아키텍처: 설계 원칙

		
증명(Attestation) 방식	생애 주기(Lineage) 추적	암호학적 서명 기반 기록
- AI 생성 코드 감지 모델의 정확도는 60~80%에 불과 - 불안정한 감지 대신 생성 시점 메타데이터 서명 기록 - 암호학적 증명을 통한 확실한 출처 보장 - 사후 감지가 아닌 생성 순간부터의 기록 중심	- 기존 정적 분석은 현재 상태(Snapshot)만 분석 - 바이브 코딩은 코드의 생애 주기 추적 필요 - 생성, 수정, 전파 전 과정을 계보로 기록 - 코드의 뿌리부터 현재까지 완전한 이력 추적	- HMAC-SHA256 서명으로 메타데이터 무결성 보장 - model_id, prompt_hash, timestamp 등 암호화 결합 - 위조 불가능한 코드 출생 증명서 발급 - 블록체인 방식의 신뢰 체계 구축

코드 블록 단위 출처 추적: GCA 메타데이터

model_id: AI 모델 및 버전

- 코드 생성에 사용된 AI 모델의 정확한 식별자 기록
- 예시: claude-sonnet-4-6, gpt-4-turbo 등
- 모델 버전별 취약점 이력 추적의 기반
- 동일 모델의 반복 패턴 분석 가능

prompt_hash: 프롬프트 SHA-256 해시

- 프롬프트 내용의 SHA-256 해시값 저장
- 프롬프트 원문은 비공개 유지 가능
- 동일 프롬프트로 생성된 코드 블록 식별
- 프롬프트-코드 인과관계 검증 근거

generation_timestamp: 생성 시각

- 코드 생성 시각을 UTC 기준으로 기록
- 예시: 2025-06-01T09:32:11Z
- 시계열 분석 및 버그 발생 시점 역추적
- 코드 수명 주기 관리의 기준점

session_id, parent_block_id, attestation_sig

- session_id: 바이브 코딩 세션 식별자
- parent_block_id: 수정 전 원본 블록 연결
- attestation_sig: 전체 필드의 HMAC 서명
- 계보 그래프 구성의 핵심 연결 정보

GCA 서명 시뮬레이터: 코드의 출생 증명서

GCA 서명 시뮬레이터

- AI가 코드를 생성하는 순간 어떤 일이 일어나는지를 직접 체험할 수 있는 시뮬레이터이다.
- AI가 생성한 코드와 사용한 모델, 세션 ID, 프롬프트를 입력하면, 메타데이터가 생성된다.
- 프롬프트 원문은 저장하지 않고, 해시값으로만 기록하기 때문에 내용은 비공개로 유지되면서도 동일한 프롬프트인지 검증 가능하다.
- 서명 생성 과정이 code_block → AST 정규화 → SHA-256 → 메타데이터 결합 → HMAC-SHA256 순서로 단계별 시각화되어 흐름을 직접 확인할 수 있다.
- unknown 모델 선택 시 서명이 생성되지 않고 UNVERIFIED로 분류되는 동작을 통해, 출처 불명 코드가 시스템에 유입될 때 어떻게 처리되는지 체험할 수 있다.

코드 블록 식별 및 출처 유형 분류

AST 정규화 후 해시 기반 block_id 생성

- Abstract Syntax Tree 파싱 후 구조 정규화
- 변수명, 공백, 주석 등 의미 무관 요소 제거
- 구조적 동일성 기준으로 고유 해시 생성
- 리팩토링 후에도 원본 블록 추적 가능

Attested AI: GCA 서명 존재하는 코드

- GCA 서명이 유효하게 존재하는 AI 생성 코드
- 모델 신뢰도와 검토 이력 기반 신뢰 평가
- 서명 검증을 통한 출처 명확성 보장
- 계보 추적이 완전하게 가능한 코드

Unattested: 서명 없이 유입된 코드

- 외부 복사 붙여넣기 등으로 유입된 코드
- GCA 서명이 없어 출처 불명 상태
- 기본적으로 낮은 신뢰 등급 부여
- 수동 검토 필수로 분류되어 경고 표시

Human Authored: 사람 검토 이력 기반 코드

- GCA 서명은 없지만 사람이 직접 작성한 코드
- 코드 리뷰 이력과 커밋 기록 기반 평가
- 검토자 수와 검토 깊이를 신뢰 근거로 활용
- 전통적 개발 방식의 신뢰 체계 유지

Git 커밋 이력과 AST 분석 결합

코드 변경의 완전한 그림을 구성하기 위해 Git 커밋 이력(변경 이력)과 AST 분석(구조 이력)을 결합합니다. 두 레이어는 독립적으로 동작하지만 block_id를 키로 연결되어 블록 수준의 시계열 계보를 완성합니다.

✓ Git Layer: 변경 이력 추적

- 커밋, 브랜치, diff 분석
- 누가, 언제 변경했는지 추적
- Git diff 파싱으로 추가/삭제 범위 추출
- 타임라인 기반 변경 이력 제공

✓ AST Layer: 구조 이력 분석

- 구문 트리, 함수/클래스 단위 분석
- 구조적 유사성 및 블록 이동 파악
- 변경 범위를 함수/클래스 단위로 분리
- 코드 의미와 구조 변화 추적



구조적 유사도 측정 및 계보 연결

Tree Edit Distance(TED) 알고리즘

- AST 노드 간 편집 거리 계산
- 구조적 동일성 기준 블록 식별
- 리팩토링 후에도 원본 추적 가능
- 유사도 공식으로 정량화

원본 블록 연결 유지

- 함수 분리, 언어 변환 후에도 연결
- 변경 전후 AST 노드 매핑
- parent_block_id로 계보 그래프 구성
- 코드 진화 과정 완전 추적

유사도 임계값 0.75 기준

- 기본 임계값 0.75로 설정
- 임계값 이상 시 동일 블록으로 판단
- 구조적 변화 허용 범위 정의
- 과도한 분리 방지

그래프 엣지 연결

- parent_block_id 필드 활용
- 변경 전후 블록을 엣지로 연결
- 방향성 그래프 구조 형성
- 계보 추적 경로 생성

위험 전파 그래프 모델링

방향성 비순환 그래프(DAG) 구조

- 코드 계보를 DAG로 모델링
- 순환 참조 없는 명확한 방향성
- 출처부터 현재까지 추적 가능
- 효율적인 그래프 탐색 지원

노드: 개별 코드 블록

- block_id로 고유 식별
- risk_score, review_status 속성 포함
- GCA 메타데이터 저장
- 신뢰 등급 정보 보유

엣지: 출처, 의존, 전파 관계

- 출처 엣지: AI 생성 → 수정본 연결
- 의존 엣지: 함수 호출 관계
- 전파 엣지: 취약점 영향 범위
- 정적 분석으로 자동 추출

영향 범위 자동 계산

- 그래프 탐색으로 영향 범위 산출
- 취약점 발견 시 의존 블록 추적
- 전파 경로 시각화 제공
- 실시간 위험 분석 수행

위험 전파 알고리즘 및 단계별 점수

취약점 발견 노드에서 시작

- 취약점 심각도를 초기 위험 점수로 설정
- 원본 블록에 1.0 점수 부여
- 그래프 탐색 시작점 정의
- 큐 기반 탐색 알고리즘 실행

의존 블록으로 위험 점수 전파 (decay=0.7)

- 의존 관계를 따라 위험 전파
- 각 단계마다 0.7 감쇠 계수 적용
- 방문한 노드 기록으로 중복 방지
- 전파 경로별 점수 계산

단계별 점수: 0단계 1.0, 1단계 0.70, 2단계 0.49

- 0단계(원본): 1.0 점수 유지
- 1단계(직접 의존): $1.0 \times 0.7 = 0.70$
- 2단계(간접 의존): $1.0 \times 0.7^2 = 0.49$
- 3단계 이상: 0.7^n 으로 계속 감소

모델 버전 단위 위험 클러스터링

- 동일 model_id 코드 블록 클러스터링
- AST 패턴으로 유사 코드 그룹화
- 클러스터 내 취약점 발견 시 전체 경고
- 모델 수준 위험 프로파일 구성

신뢰 점수 산출 알고리즘

네 가지 요소의 가중 합산

- $T(b) = w1 \times \text{Attestation} + w2 \times \text{ModelTrust} + w3 \times \text{Review} + w4 \times (1 - \text{PropagatedRisk})$
- 각 요소별 가중치 적용
- 0~1 범위의 정규화된 점수
- 설명 가능한 신뢰 근거 제공

Attestation(25%): GCA 서명 유효성

- 유효한 서명: 1.0 점수
- 서명 없음: 0.2 점수
- 위조된 서명: 0.0 점수
- 암호학적 검증 기반 평가

ModelTrust(25%): 모델 신뢰 등급

- 생성 모델의 신뢰 등급 반영
- 모델별 취약점 발생 이력 분석
- 0~1 범위로 정규화
- 모델 버전별 차등 적용

Review(30%) 및 PropagatedRisk(20%)

- Review: 검토자 수, 최신성, 깊이 반영
- PropagatedRisk: 전파 알고리즘 결과
- 사람 검토가 가장 높은 가중치
- 의존 블록 위험도 고려

신뢰 등급 분류 및 설명 가능한 컨텍스트

TRUSTED (0.80 이상): 검증된 코드 - 완전히 검증된 신뢰 가능 코드 - 정기 재검토 권장 - 프로덕션 배포 승인 - 안정적인 운영 가능	REVIEWED (0.60~0.79): 재확인 권장 - 검토되었으나 완전하지 않음 - 배포 전 재확인 필요 - 추가 검토로 등급 상승 가능 - 조건부 배포 허용
UNVERIFIED (0.40~0.59): 수동 검토 필수 - 검토 능력 부족한 코드 - 수동 검토 필수 요구 - 배포 전 보안 점검 필요 - 위험 요소 확인 후 결정	RISKY (0.39 이하): 즉시 격리 및 검토 - 출처 불명 또는 위험 전파 - 즉시 격리 및 상세 검토 - 배포 차단 조치 - 전면 재작성 고려 필요

신뢰 점수 계산기: 점수의 근거



신뢰 점수 계산기

- 코드 블록 하나의 신뢰 점수가 어떻게 산출되는지 계산하는 계산기이다.
- 슬라이더 네 개가 GCA 서명 유효성, 모델 신뢰 등급, 사람 검토 이력, 전파된 위험도를 나타낸다.
- 검토 이력의 가중치가 30%로 가장 높는데, 이는 AI가 생성했더라도 사람이 충분히 검토한 코드는 높은 신뢰를 받을 수 있다는 설계 철학이 반영된 것이다.
- 전파 위험도는 점수를 깎는 방향으로 작용한다.코드 자체에는 문제가 없어도, 의존하는 블록에 취약점이 있으면 등급이 내려갑니다.

전체 시스템 컴포넌트

본 시스템은 바이브 코딩 환경에서 AI 생성 코드의 출처와 생성 맥락을 추적하여 신뢰 수준을 정량화하는 계보 기반 분석 플랫폼입니다. 각 컴포넌트는 독립적으로 동작하면서도 유기적으로 연결되어 코드의 생애 주기 전반을 추적하고 검증합니다.



✓ GCA Injector

- 코드 생성 시 GCA 메타데이터 서명 삽입
- LLM API Hook 및 HMAC-SHA256 암호화

✓ AST Parser

- 코드 블록 추출 및 AST 정규화
- tree-sitter 기반 다언어 지원

✓ Lineage DB

- 계보 그래프 저장 및 질의
- Neo4j 그래프 데이터베이스 활용

✓ Risk Propagator

- 취약점 전파 경로 자동 계산
- Python 및 NetworkX 기반 분석

✓ Trust Scorer & IDE Plugin

- 신뢰 점수 산출 및 레포트 생성
- VS Code Extension API 통합

기존 도구 대비 차별점

분석 관점: Snapshot에서 Lineage로 전환

- 기존 도구(Snyk, CodeQL)는 현재 상태만 분석
- 본 시스템은 코드의 생애 주기 전체를 추적
- 변경 이력뿐 아니라 생성 의도까지 파악
- 시간 흐름에 따른 코드 진화 과정 시각화

AI 코드 처리: GCA 서명 기반 생성 맥락 추적

- 기존 도구는 AI/인간 코드를 구분하지 않음
- GCA 서명으로 생성 맥락을 암호학적으로 기록
- 모델 ID, 프롬프트 해시, 생성 시각 추적
- 감지가 아닌 증명 기반 접근 방식

취약점 분석: 모델 버전 단위 패턴 클러스터링

- 개별 파일 단위 스캔을 넘어선 분석
- 동일 모델이 반복 생성하는 취약 패턴 클러스터링
- 모델 수준의 위험 프로파일 구성
- 패턴 기반 사전 예방적 경고 시스템

바이브 코딩 특화: 세션 단위 계보 추적 및 프롬프트 연결

- 바이브 코딩 세션을 추적 단위로 정의
- 프롬프트-코드 인과관계 명확히 연결
- 세션 단위 계보 그래프 시각화 제공
- 버그 발생 시 원인 세션 즉시 파악 가능

실제 적용 방식 (1): 개발 단계

IDE 플러그인을 통한 GCA 서명 자동 부여

- Cursor, VS Code 등 IDE에서 자동 동작
- AI 코드 생성 즉시 백그라운드에서 서명 부여
- 개발자의 별도 조작 없이 투명하게 작동
- 모델 ID, 세션 ID, 생성 시각 자동 기록

사이드패널에 신뢰 등급 색상 표시

- 코드 블록별 신뢰 등급을 색상으로 시각화
- TRUSTED(녹색), REVIEWED(노란색), UNVERIFIED(주황색), RISKY(빨간색)
- 검토 이력 없는 AI 생성 블록 별도 하이라이팅
- 실시간 신뢰 점수 변화 모니터링

AST 비교로 원본 블록 연결 유지

- 코드 편집 시 AST 구조 비교 자동 실행
- Tree Edit Distance 알고리즘으로 유사도 측정
- 리팩토링 후에도 원본 블록과 계보 연결
- parent_block_id 필드로 변경 이력 추적

검토 이력 기록 및 신뢰 점수 실시간 업데이트

- 파일 저장 시 신뢰 점수 자동 재계산
- 코드 리뷰 완료 시 검토 이력 기록
- REVIEWED 등급으로 자동 상향 조정
- 사이드패널에 변경 사항 즉시 반영

실제 적용 방식 (2): 통합 단계

Git PR 과정에 자동 검증 삽입 - Pull Request 생성 시 자동 트리거 - 변경된 코드 블록의 신뢰 등급 자동 분석 - 기존 워크플로우에 자연스럽게 통합 - 개발자의 추가 학습 부담 최소화	CI/CD 파이프라인에서 계보 분석 실행 - CI/CD 단계에서 계보 그래프 분석 자동 실행 - 전체 코드베이스의 신뢰도 검증 - 의존성 체인 전체의 위험 전파 경로 추적 - 분석 결과를 PR 코멘트로 자동 첨부
RISKY/UNVERIFIED 블록 포함 시 병합 차단 - RISKY 등급 블록 포함 시 PR 자동 차단 - UNVERIFIED 블록 포함 시 경고 표시 - 병합 전 필수 검토 요구 사항 설정 - 신뢰 점수 임계값 팀별 커스터마이징 가능	경고 발생 및 검토 요청 알림 - 위험 블록 발견 시 담당자에게 알림 발송 - 구체적인 위험 요소와 권장 조치 제공 - 검토 완료 후 재분석 자동 실행 - 승인 프로세스 자동화로 병목 현상 방지

실제 적용 방식 (3): 운영 단계

취약점 발생 시 계보 그래프 역추적

- 프로덕션 환경 장애 발생 시 즉시 대응
- 계보 그래프를 역방향으로 자동 탐색
- 문제 코드의 생성 세션과 프롬프트 추적
- 수동 탐색 대비 시간 대폭 단축

원인 블록 즉시 특정

- 장애 원인 코드 블록 즉시 식별
- GCA 서명으로 생성 맥락 확인
- 어떤 모델이 언제 어떤 의도로 생성했는지 파악
- 평균 4~8시간 소요 작업을 30분 이내로 단축

전파 그래프로 영향 범위 자동 산출

- DAG 기반 의존성 그래프 자동 분석
- 취약점이 전파된 모든 블록 자동 추출
- 전파 단계별 위험 점수 계산(decay=0.7)
- 영향받는 서비스 범위 즉시 파악

동일 모델 패턴 전체 경고

- 동일 모델 버전이 생성한 유사 패턴 클러스터링
- AST 구조 기반 패턴 매칭 자동 실행
- 잠재적 위험 블록 사전 경고
- 모델 수준의 위험 프로파일 업데이트

기대 효과

장애 원인 파악 시간: 4~8시간 → 30분 이내

- 계보 그래프 역추적으로 즉시 원인 특정
- 수동 코드 탐색 시간 90% 이상 절감
- 장애 대응 속도 획기적 개선
- 서비스 다운타임 최소화

미검토 AI 코드 비율 실시간 가시화

- 전체 코드베이스의 신뢰도 실시간 모니터링
- UNVERIFIED 블록 비율 대시보드 제공
- 팀별, 프로젝트별 신뢰도 추이 분석
- 데이터 기반 코드 품질 관리 가능

보안 취약점 사전 차단 강화

- 계보 기반 예측적 취약점 탐지
- 모델 패턴 클러스터링으로 사전 경고
- CI/CD 단계에서 위험 블록 자동 차단
- 프로덕션 배포 전 보안 검증 강화

신규 입사자 코드 파악 시간 대폭 단축

- 코드 생성 맥락과 계보 그래프 제공
- 수년 전 코드도 생성 의도 즉시 파악
- 온보딩 기간 단축 및 학습 곡선 완화
- 인수인계 시 정보 손실 최소화

한계점: 솔직한 문제 인식

초기 데이터 수집의 어려움

- GCA 서명은 생성 시점부터 부여되어야 완전히 작동
- 도입 이전 레거시 코드, 외부 복사 코드는 서명 없음
- 초기 도입 시 전체 코드베이스가 UNVERIFIED 상태
- 신뢰도가 실제로 쌓이기까지 일정 도입 기간 필요
- **대응:** 수동 검토 후 일괄 등급 부여로 점진적 전환

AST 기반 추적의 기술적 한계

- 코드가 다른 언어로 전환되거나 전면 아키텍처 개편 시 계보 연결 끊길 수 있음
- Tree Edit Distance 계산은 코드베이스 규모 커질수록 연산 비용 증가
- **대응:** 주요 언어 우선 지원 후 단계적 확장

GCA 생태계 협력 의존

- Cursor, GitHub Copilot 등 주요 도구의 GCA 연동 지원 필요
- 초기에는 플러그인 방식으로 우회 가능하나 완성도 한계
- **대응:** 오픈 표준으로 제안하여 생태계 확산 유도

신뢰 점수 오남용 가능성

- 높은 점수 = 무조건 안전, 낮은 점수 = 무조건 배제 착각 발생 가능
- 점수 관리를 위한 형식적 검토 문화 생길 위험
- **대응:** 점수는 판단 보조 수단임을 명시, 운영 가이드라인 병행

확장 가능성: 코드를 넘어서

멀티 에이전트 환경으로의 확장

- 현재: 사람 ↔ AI 1대1 구조
- 가까운 미래: 여러 AI 에이전트가 협력해 코드 생성
- agent_id 필드 추가만으로 멀티 에이전트 계보 추적 자연스럽게 확장
- 다중 출처 계보 네트워크로 진화

SBOM 국제 표준 확장안 제안

- 현재 SBOM은 라이브러리·의존성 단위 출처 기록
- 본 시스템의 GCA 서명 체계 = SBOM을 코드 블록 단위로 세분화한 확장판
- NIST, CISA 등 국제 표준 기구에 AI 생성 코드 추적 표준안으로 제안 가능

EU AI Act 컴플라이언스 인프라

- EU AI Act는 고위험 AI 시스템의 투명성·추적 가능성 요구
- 계보 추적 이력 → AI 코드 사용 내역 증명하는 컴플라이언스 문서로 직접 활용
- 보안 도구가 아닌 **규제 대응 인프라**로 포지셔닝

AI 생성 산출물 전반으로 확장

- 코드 → 인프라 설정(IaC) → AI 생성 문서 → 테스트 케이스 → 프롬프트 자체
- 소프트웨어 전체 생애 주기에서 AI 개입 산출물의 출처·맥락 추적 플랫폼으로 진화